Microsoft SQL Server

Petrescu Alexandru



Overview

- About SQL Server 2017
- Services & Management Tools
- SQL & T-SQL
- Installing
- Working with T-SQL



SQL Server 2017

Overview



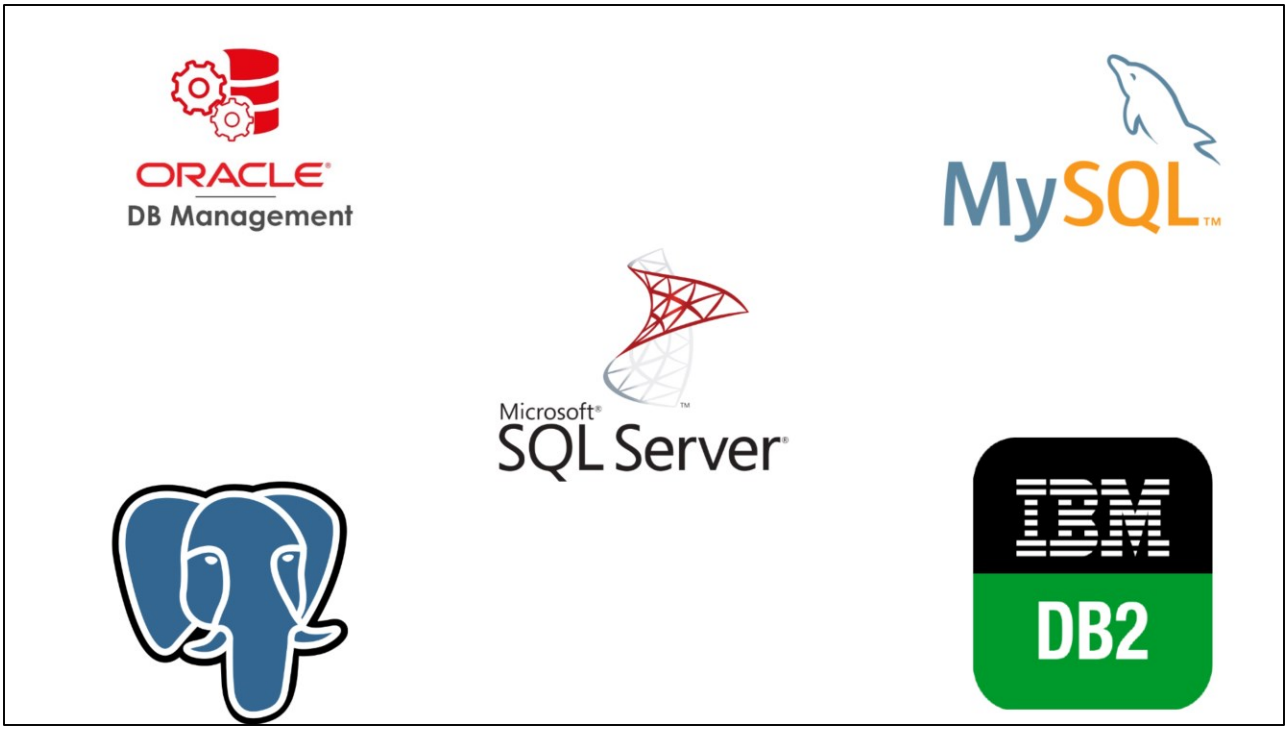
RDBMS

- The **relational** model organizes data into one or more **tables**, where **columns** represent **attributes** of the “entity” and **rows** are **instances** of it. This model accepts only **structured** data where each entity must fit a certain pattern.

- SQL Server 2017 (from Microsoft) is a Relational Database Management System (**RDBMS**) capable of processing large amounts of data preparing for the big-data era.



General info about relational databases, how an entity can be mapped to an object in the OOP.



https://db-engines.com/en/ranking/relational+dbms

DB-Engines Ranking of Relational DBMS

Rank Sep 2019	Rank Aug 2019	Rank Sep 2018	DBMS	Database Model	Score Sep 2019	Score Aug 2019	Score Sep 2018
1.	1.	1.	Oracle	Relational, Multi-model	1346.66	+7.18	+37.54
2.	2.	2.	MySQL	Relational, Multi-model	1279.07	+25.39	+98.60
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	1085.06	-8.12	+33.78
4.	4.	4.	PostgreSQL	Relational, Multi-model	482.25	+0.91	+75.82
5.	5.	5.	IBM Db2	Relational, Multi-model	171.56	-1.39	-9.50
6.	6.	6.	Microsoft Access	Relational	132.71	-2.63	-0.69
7.	7.	7.	SQLite	Relational	123.36	+0.65	+7.91
8.	8.	↑ 9.	MariaDB	Relational, Multi-model	86.07	+1.11	+15.43
9.	9.	↑ 10.	Hive	Relational	83.10	+1.30	+23.46
10.	10.	↓ 8.	Teradata	Relational, Multi-model	76.97	+0.32	-0.42

History

- The history of Microsoft SQL Server begins with the first Microsoft SQL Server product—SQL Server 1.0, a 16-bit server for the OS/2 operating system in 1989—and extends to the current day.

- Microsoft's latest stable SQL Server is the 2017th edition, while their 2019th edition is still in preview but with the promise of providing key elements of a **data lake**—Hadoop Distributed File System (HDFS), Spark, and analytics tools.



Development Process

- It has been developed entirely by Microsoft since the beginning and is **closed-source**, but with the new **open-source** curent it can shift to having its code available in Git.

- It is written in C/C++ like most of the Microsoft (core) products.



Editions

- It offered multiple editions throughout history. Today it has 4 editions that have differences only in scalability, transparency, and performance:
  - **Enterprise**, best for business - unlimited scaling
  - **Standard**, ideal for small business - moderate scaling
  - **Express**, portable version - lowest scaling
  - **Developer**, non-commercial and it can scale as much as the machine



OS/Cloud Availability

- It now runs on **Windows**, **Linux**, and **Docker**, but not any distribution or version. Since it is **closed-source**, you can not compile it for yourself. Linux list of support :
  - Red Hat Enterprise 7.3
  - Ubuntu Linux 16.04
  - SUSE Linux Enterprise Server v12 SP2
- It is **cloud-ready** and offered as Database-As-A-Service (**Daas**, part of Platform-As-A-Service) on multiple cloud providers and in Azure (Microsoft's Cloud Platform) it has a special flavor called Azure SQL Database.



For Developers

- It has native connectors, code-side compatibility and (sometimes) Object-relational mapping (**ORM**) for multiple languages such as:
  - C#
  - Java
  - NodeJS
  - PHP
  - Python
  - Ruby
  - (Google) GO



Performance

- **Columnstore indexes** accelerate analytics performance by redefining data storage and queries.
- **In-Memory OLTP** achieves greater speed through memory-optimized tables.
- **Adaptive Query Processing** shapes optimization strategies based on application runtime conditions and characteristics.
- **Automatic Database Tuning** corrects query execution plans causing performance issues.
- **Query Store** gives insights on query plan choice and performance.



Availability

- **Always On Failover Cluster Instances and Availability Groups** enable HADR scenarios on both Linux and Windows.
- **Enhanced Always On** offers up to eight secondary replicas per primary database, plus a Replica Wizard.
- **Read-Scale Availability Groups** provide clusterless support for read-only workloads.



HADR = high availability disaster recovery

Security

- **Always Encrypted** provides separation between those who own and manage data.
- **Dynamic Data Masking** limits data exposure while simplifying security design and coding.
- **Row-Level Security** controls access to rows in a database table based on user characteristics.
- **Backup encryption** gives extra security for backup files.



HADR = high availability disaster recovery

Replication

- **Replication** is a set of technologies for **copying** and **distributing** data and database objects from one database to another and then synchronizing to maintain **consistency**.

  Replication to SQLCE is supported on Windows Server 2012 and Windows 8. As an alternative to replication, there is Microsoft Sync Framework which includes a flexible API that make it easy to synchronize among SQL Server, SQL Server Express, SQL Server Compact, and SQL Azure databases.



SQL Server Compact is a compact version designed for mobile devices, similar to devexpress.

Sharding

- **Sharding** is a technique to **distribute** large amounts of identically structured data across a number of **independent** databases. It is especially popular with cloud developers creating Software as a Service (**SAAS**) offerings for end customers or businesses.

- It is not directly available for SQL Server 2017 but it is for its Azure flavor, Azure Database.



Query Processing

- SQL Server Database Engine processes queries on various data storage architectures such as **local** tables, **partitioned** tables, and tables **distributed** across multiple servers
- Query **optimization** for reused code goes through **execution plan caching**



Execution modes

- The SQL Server Database Engine can process Transact-SQL statements using two distinct processing modes:
  - **Row** mode execution
  - **Batch** mode execution

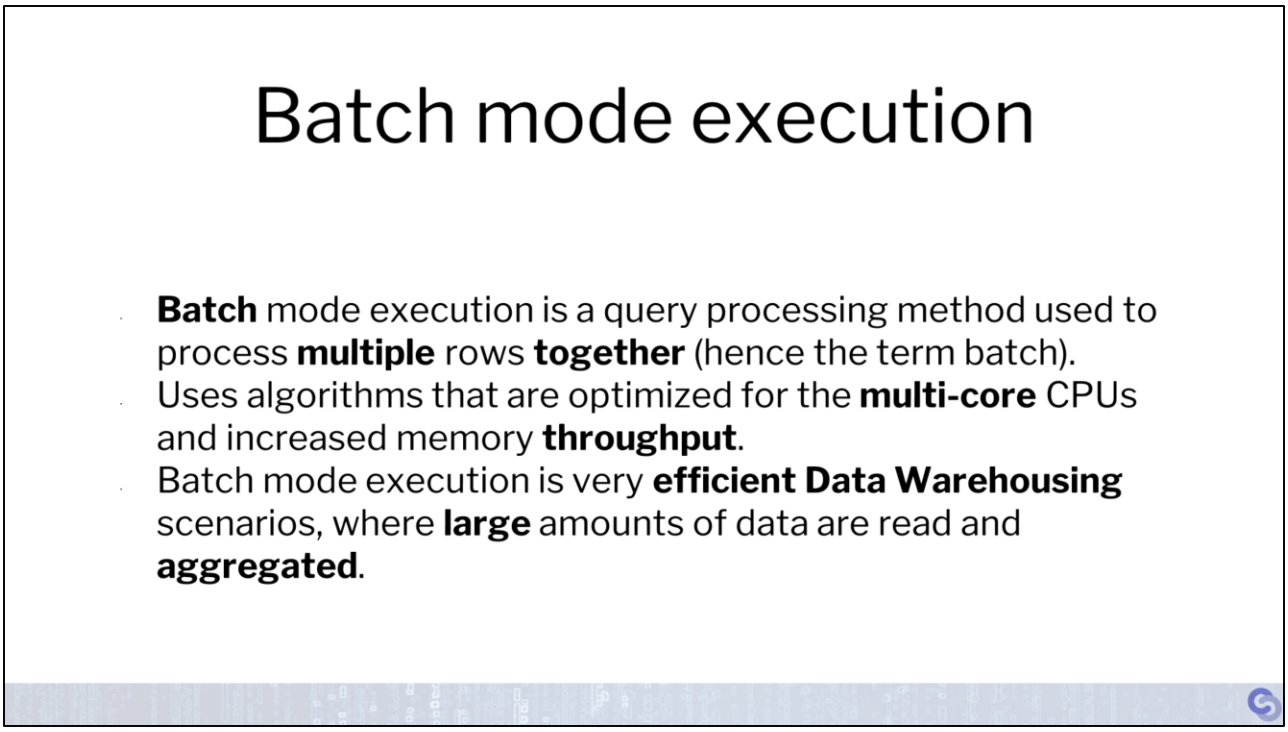


Row mode execution

- **Row** mode execution is a query processing method used with traditional **RDBMS** tables, where data is stored in row format.
- Row mode execution is very **efficient** for **OLTP** scenarios, but can be **less efficient** when scanning large amounts of data, for example in **Data Warehousing** scenarios.



When a query is executed and accesses data in row store tables, the execution tree operators and child operators read each required row, across all the columns specified in the table schema. From each row that is read, SQL Server then retrieves the columns that are required for the result set, as referenced by a SELECT statement, JOIN predicate, or filter predicate.



Each column within a batch is stored as a vector in a separate area of memory, so batch mode processing is vector-based.

Batch mode execution is closely integrated with, and optimized around, the columnstore storage format. Batch mode processing operates on compressed data when possible, and eliminates the exchange operator used by row mode execution. The result is better parallelism and faster performance.

When a query is executed in batch mode, and accesses data in columnstore indexes, the execution tree operators and child operators read multiple rows together in column segments. SQL Server reads only the columns required for the result, as referenced by a SELECT statement, JOIN predicate, or filter predicate.

Storage

- The **page** is the **fundamental** unit of data storage.
- An **extent** is a collection of **eight physically contiguous** pages. Extents help efficiently manage pages.
- The disk space allocated to a **data file** (.mdf/.ndf) in a DB is logically divided into **pages** numbered contiguously.
- Disk I/O operations are performed at the **page level**. All pages are stored in extents.



Pages

- In SQL Server, the **page** size is **8-KB**. This means a SQL Server databases has 128 pages per megabyte.
- Each page begins with a **96-byte header** that is used to store system information about the page, including: page **number**, page **type**, the amount of **free space** on the page, and the allocation unit **ID** of the object that owns the page.



Log files do not contain pages; they contain a series of log records.

Extents

- Extents are the basic unit in which space is managed. An extent is eight physically contiguous pages, or 64 KB.
- SQL Server has two types of extents:
  - Uniform extents: are owned by a single object; all eight pages in the extent can only be used by the owning object.
  - Mixed extents: are shared by up to eight objects. Each of the eight pages in the extent can be owned by a different object.



Up to, and including, SQL Server 2014 (12.x), trace flag 1118 can be used to change the default allocation to always use uniform extents. For more information about this trace flag, see DBCC TRACEON - Trace Flags.

Starting with SQL Server 2016 (13.x), the functionality provided by TF 1118 is automatically enabled for TempDB. For user databases, this behavior is controlled by the SET MIXED_PAGE_ALLOCATION option of ALTER DATABASE, with the default value set to OFF, and trace flag 1118 has no effect. For more information, see ALTER DATABASE SET Options (Transact-SQL).

SQL Server 2017

Services & Management Tools



Services & Tolls

- There are multiple services that can be installed for SQL Server 2017 adding more features to the classical RDBMS

- There are also multiple management tools, with different features that simplify the process of dealing with a database, but not using them does not mean that the terminal will not be able to use the services



SQL Server Database Engine

- Is the core service for SQL Server and the only one mandatory to use this **RDBMS**.

- It includes the Database Engine, the core service for storing, processing, and securing data, replication, full-text search, tools for managing relational and **XML** data, in database analytics integration, and **PolyBase** integration for access to Hadoop and other heterogeneous data sources, and the Data Quality Services (**DQS**) server.



Analysis Services

- Analysis Services includes the tools for creating and managing online analytical processing (**OLAP**) and data mining applications.



Reporting Services

Reporting Services includes server and client components for creating, managing, and deploying tabular, matrix, graphical, and free-form reports. Reporting Services is also an extensible platform that you can use to develop report applications.



Integration Services

Integration Services is a set of graphical tools and programmable objects for moving, copying, and transforming data. It also includes the Data Quality Services (**DQS**) component for Integration Services.



Master Data Services

Master Data Services (**MDS**) is the SQL Server solution for master data management. MDS can be configured to manage any domain (products, customers, accounts) and includes hierarchies, granular security, transactions, data versioning, and business rules, as well as an Add-in for Excel that can be used to manage data.



Machine Learning Services

Machine Learning Services (In-Database) supports distributed, scalable machine learning solutions using enterprise data sources. In SQL Server 2016, the R language was supported. SQL Server 2017 supports R and Python.



Machine Learning Server

Machine Learning Server (Standalone) supports deployment of distributed, scalable machine learning solutions on multiple platforms and using multiple enterprise data sources, including Linux and Hadoop. In SQL Server 2016, the R language was supported. SQL Server 2017 supports R and Python.



Management Tools

- SQL Server Management Studio (Windows Only)
- SQL Server Configuration Manager
- SQL Server Profiler
- Database Engine Tuning Advisor
- Data Quality Client
- SQL Server Data Tools
- Connectivity Components



SQL Server Management Studio is an integrated environment to access, configure, manage, administer, and develop components of SQL Server. Management Studio lets developers and administrators of all skill levels use SQL Server.
SQL Server Configuration Manager provides basic configuration management for SQL Server services, server protocols, client protocols, and client aliases.
SQL Server Profiler provides a graphical user interface to monitor an instance of the Database Engine or Analysis Services.
Database Engine Tuning Advisor helps create optimal sets of indexes, indexed views, and partitions.
Provides a highly simple and intuitive graphical user interface to connect to the DQS server, and perform data cleansing operations. It also allows you to centrally monitor various activities performed during the data cleansing operation.
SQL Server Data Tools provides an IDE for building solutions for the Business Intelligence components: Analysis Services, Reporting Services, and Integration Services.

(Formerly called Business Intelligence Development Studio).

SQL Server Data Tools also includes "Database Projects", which provides an integrated environment for database developers to carry out all their database design work for any SQL Server platform (both on and off premise) within Visual Studio. Database developers can use the enhanced Server Explorer in Visual Studio to easily create or edit database objects and data, or execute queries.

Installs components for communication between clients and servers, and network libraries for DB-Library, ODBC, and OLE DB.

SQL Server 2017

SQL & T-SQL



Transactions

- A transaction is a single unit of work.

- If a transaction is successful, all of the data modifications made during the transaction are committed and become a permanent part of the database. If a transaction encounters errors and must be canceled or rolled back, then all of the data modifications are erased.



Autocommit transactions

- Each individual statement is a transaction.



Explicit transactions

- Each transaction is explicitly started with the BEGIN TRANSACTION statement and explicitly ended with a COMMIT or ROLLBACK statement.



Implicit transactions

- A new transaction is implicitly started when the prior transaction completes, but each transaction is explicitly completed with a COMMIT or ROLLBACK statement.



Batch-scoped transactions

Applicable only to multiple active result sets (MARS), a Transact-SQL explicit or implicit transaction that starts under a MARS session becomes a batch-scoped transaction. A batch-scoped transaction that is not committed or rolled back when a batch completes is automatically rolled back by SQL Server.



Data Representation

DataTypes in T-SQL

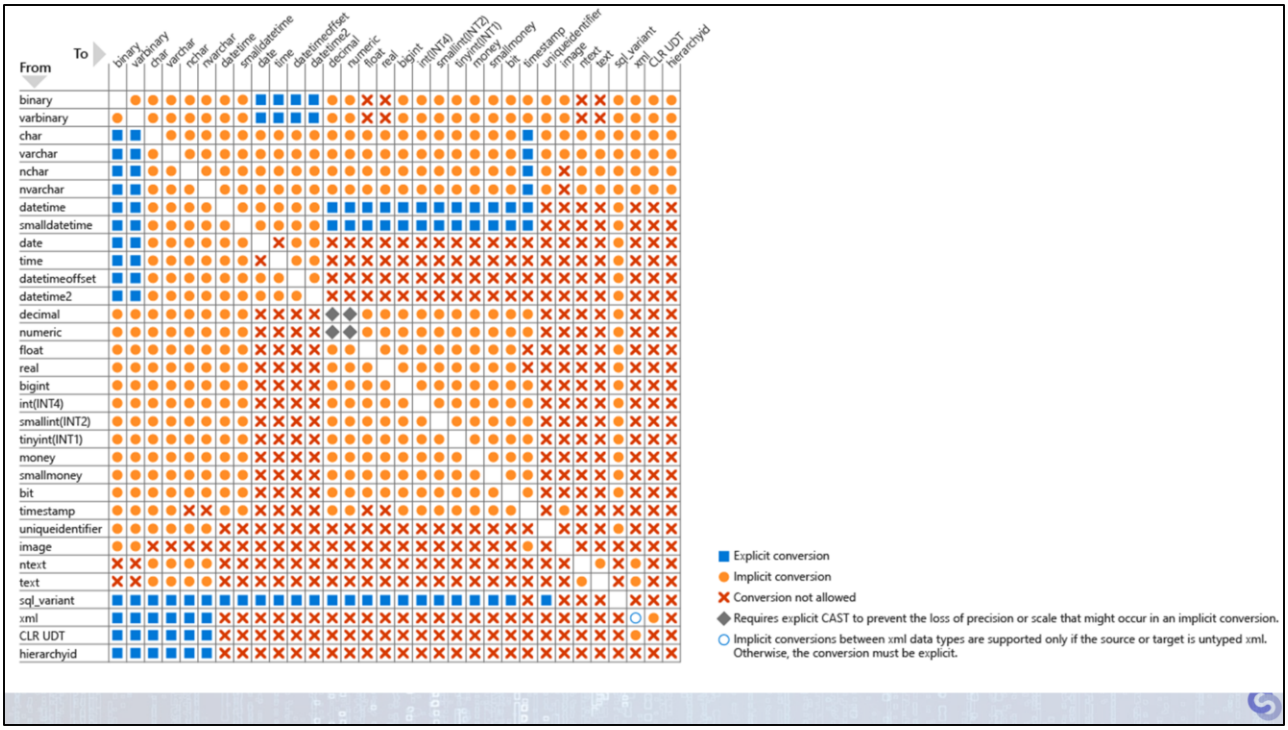
SQL Server supplies a set of system data types that define all the types of data that can be used with SQL Server. You can also define your own data types in T-SQL or the Microsoft .NET Framework. **Alias** data types are based on the system-supplied data types. **User-defined** types obtain their characteristics from the methods and operators of a class that you create by using one of the programming languages support by the .NET Framework.



DataTypes in T-SQL

- Data types in SQL Server are organized into the following categories:
  - Exact numerics
  - Unicode character strings
  - Approximate numerics
  - Binary strings
  - Date and time
  - Other data types




From
To
binary
varbinary
char
varchar
nchar
nvarchar
datetime
smalldatetime
date
time
datetimeoffset
datetime2
decimal
numeric
float
real
bigint
int(INT4)
smallint(INT2)
tinyint(INT1)
money
smallmoney
bit
timestamp
uniqueidentifier
image
ntext
text
sql_variant
xml
CLR UDT
hierarchyid
Explicit conversion
Implicit conversion
Conversion not allowed
Requires explicit CAST to prevent the loss of precision or scale that might occur in an implicit conversion.
Implicit conversions between xml data types are supported only if the source or target is untyped xml. Otherwise, the conversion must be explicit.

Variables

- Variables can be of any type: standard/table
- DECLARE @variable_name <var_type> [= <initial_value>]
- SET/SELECT

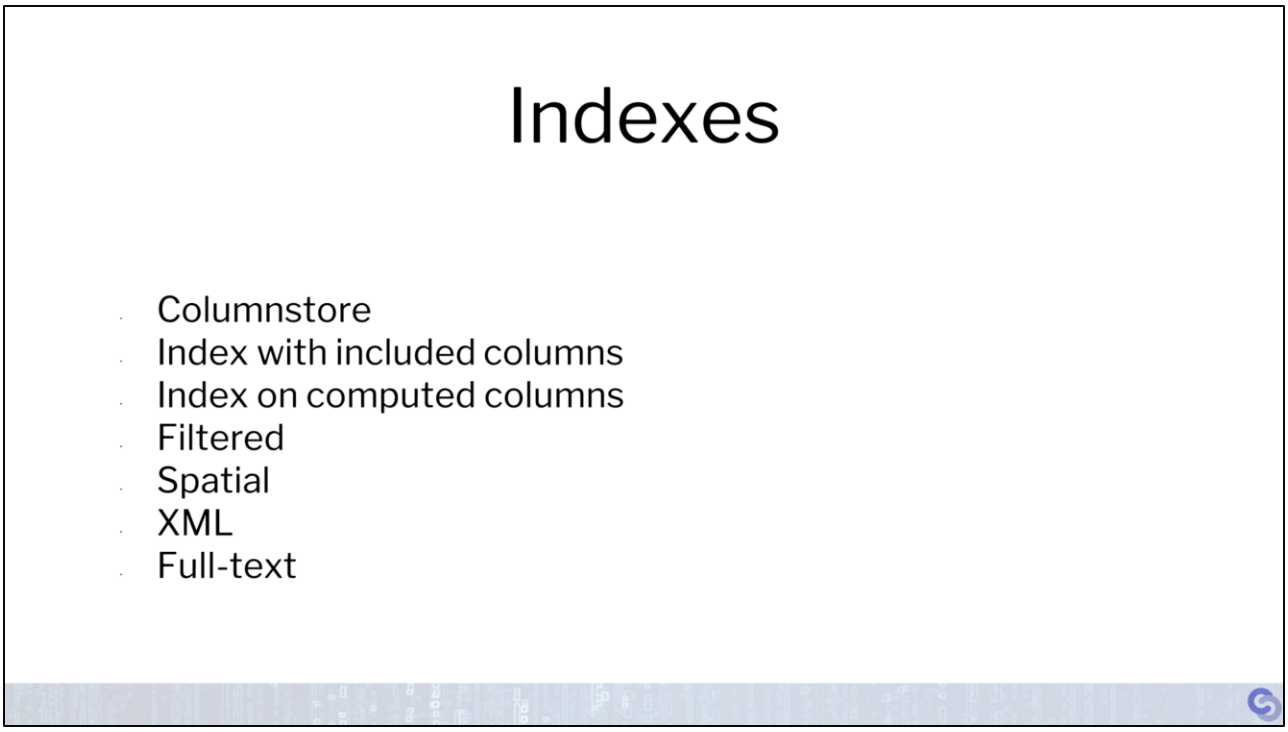


Indexes

- Hash
- Nonclustered (memory-optimized)
- Nonclustered
- Clustered
- Unique



With a hash index, data is accessed through an in-memory hash table. Hash indexes consume a fixed amount of memory, which is a function of the bucket count.
For memory-optimized nonclustered indexes, memory consumption is a function of the row count and the size of the index key columns.
A nonclustered index can be defined on a table or view with a clustered index or on a heap. Each index row in the nonclustered index contains the nonclustered key value and a row locator. This locator points to the data row in the clustered index or heap having the key value. The rows in the index are stored in the order of the index key values, but the data rows are not guaranteed to be in any particular order unless a clustered index is created on the table.
A clustered index sorts and stores the data rows of the table or view in order based on the clustered index key. The clustered index is implemented as a B-tree index structure that supports fast retrieval of the rows, based on their clustered index key values.
A unique index ensures that the index key contains no duplicate values and therefore every row in the table or view is in some way unique. Uniqueness can be a property of both clustered and nonclustered indexes.



An in-memory columnstore index stores and manages data by using column-based data storage and column-based query processing. Columnstore indexes work well for data warehousing workloads that primarily perform bulk loads and read-only queries. Use the columnstore index to achieve up to 10x query performance gains over traditional row-oriented storage, and up to 7x data compression over the uncompressed data size.

A nonclustered index that is extended to include nonkey columns in addition to the key columns.

An index on a column that is derived from the value of one or more other columns, or certain deterministic inputs.

An optimized nonclustered index, especially suited to cover queries that select from a well-defined subset of data. It uses a filter predicate to index a portion of rows in the table. A well-designed filtered index can improve query performance, reduce index maintenance costs, and reduce index storage costs compared with full-table indexes.

A spatial index provides the ability to perform certain operations more efficiently on spatial objects (spatial data) in a column of the geometry data type. The spatial index reduces the number of objects on which relatively costly spatial operations need to be applied.

A shredded, and persisted, representation of the XML binary large objects (BLOBs) in the xml data type column.

A special type of token-based functional index that is built and maintained by the Microsoft Full-Text Engine for SQL Server. It provides efficient support for sophisticated word searches in character string data.

Views

- A view is a virtual table whose contents are defined by a query. Like a table, a view consists of a set of named columns and rows of data.

- Unless indexed, a view does not exist as a stored set of data values in a database. The rows and columns of data come from tables referenced in the query defining the view and are produced dynamically when the view is referenced.



Types of Views

- Indexed Views
- Partitioned Views
- System Views



Temporary Tables

- They exist only during the current **session** (**connection**)

- They can be either for the one **who created** them (#temporaryTable) or they can be **shared** during that session (##globalTemporaryTable)



Types of Queries

Dynamic SQL

- Not so fast, but can be anything
- Usually used with Cursors
- EXEC @some_sql



Cursor in T-SQL

- A **cursor** is a data type for variables or stored procedure **OUTPUT** parameters that contain a reference to a cursor.
- The operations that can reference variables and parameters having a cursor data type are:
  - **DECLARE** and **SET** *@local_variable* statements.
  - **OPEN**, **FETCH**, **CLOSE**, and **DEALLOCATE** cursor statements.
  - Stored procedure output parameters.
  - CURSOR_STATUS function.
  - sp_cursor_list, sp_describe_cursor, sp_describe_cursor_tables, and sp_describe_cursor_columns system stored procedures.



XQuery

- Transact-SQL supports a subset of the XQuery language that is used for querying the xml data type aligned with the July 2004 Working Draft of XQuery. The language is under development by the World Wide Web Consortium (W3C), with the participation of all major database vendors and also Microsoft.
- **XQuery** is a language that can query structured or semi-structured XML data. With the xml data type support provided in the Database Engine, documents can be stored in a database and then queried by using XQuery.



XML DML

- The XML Data Modification Language is an extension of the XQuery language. As defined by W3C, the XQuery language lacks the Data Manipulation (DML) part. The XML DML introduced in this topic, and also the XQuery language, provides a fully functional query and data-modification language that you can use against the xml
- It adds the following case-sensitive keywords to XQuery:
  - Insert
  - Delete
  - Replace value of



GRAPH QUERY

- Graph databases represent relationships (**edges**) between entities (**nodes**).
- Graph databases make finding and expressing relationships easier, helping to generate better insights from interconnected data.



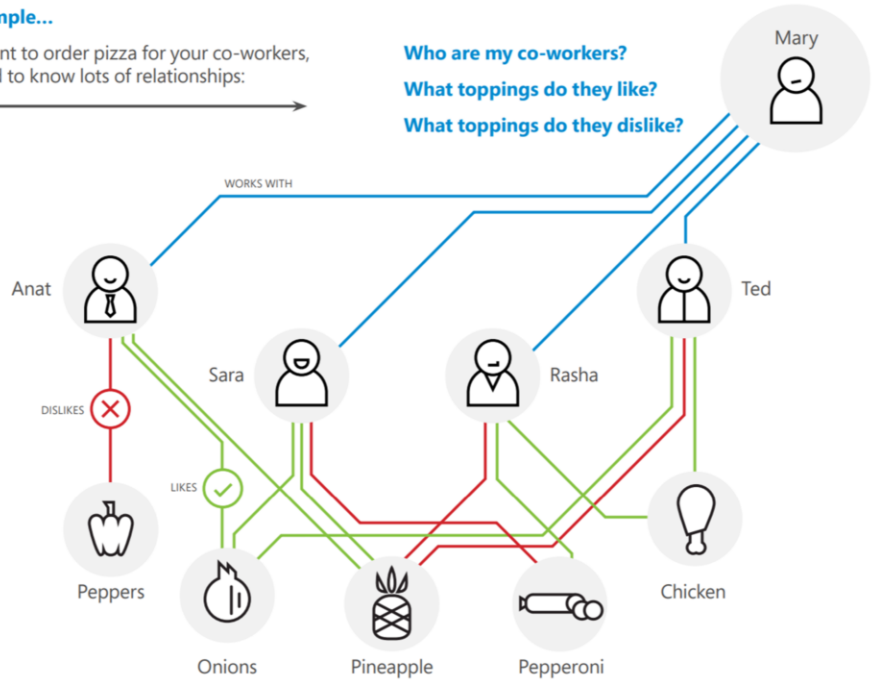
For example...

If you want to order pizza for your co-workers, you need to know lots of relationships:

Who are my co-workers?

What toppings do they like?

What toppings do they dislike?



TRADITIONAL SQL QUERY: VS **GRAPH QUERY:**

```
SELECT DISTINCT Toppings.topping
FROM dbo.toppings AS Toppings
WHERE Toppings.id IN
      (SELECT Likes.toppingID
      FROM dbo.relToppingLikes AS Likes
      WHERE Likes.peopleID IN
            (SELECT Coworkers.coworkerID
            FROM dbo.relCoworkers AS Coworkers
            WHERE Coworkers.personID IN
                  (SELECT Person.id
                  FROM dbo.people AS Person
                  WHERE Person.firstName = 'Mary')))
      AND Toppings.id NOT IN
      (SELECT DisLikes.toppingID
      FROM dbo.relToppingDisLikes AS DisLikes
      WHERE DisLikes.peopleID IN
            (SELECT Coworkers.coworkerID
            FROM dbo.relCoworkers AS Coworkers
            WHERE Coworkers.personID IN
                  (SELECT Person.id
                  FROM dbo.people AS Person
                  WHERE Person.firstName = 'Mary')))
```

```
SELECT DISTINCT Topping.topping
FROM gPeople AS Person, gPeople AS Coworker,
eWorkswith, eLikes, eDisLikes, gToppings AS Topping
WHERE MATCH(Person-(eWorkswith)->Coworker-(eLikes)-
>Topping)
AND Topping.ID NOT IN
      (SELECT DISTINCT Topping.ID
      FROM gPeople AS Coworker, eWorkswith,
      eDisLikes, gToppings AS Topping
      WHERE MATCH(Person-(eWorkswith)->Coworker-
      (eDisLikes)->Topping))
AND Person.firstName='Mary'
```

Instructions

Instruction Types in T-SQL

- Data Definition Language (**DDL**): **CREATE**, **ALTER**, **DROP**
- Data Control Language (**DCL**): **GRANT**, **DENY**, **REVOKE**
- Data Manipulation Language (**DML**):**SELECT**, **INSERT**, **UPDATE**, **DELETE**, **MERGE**

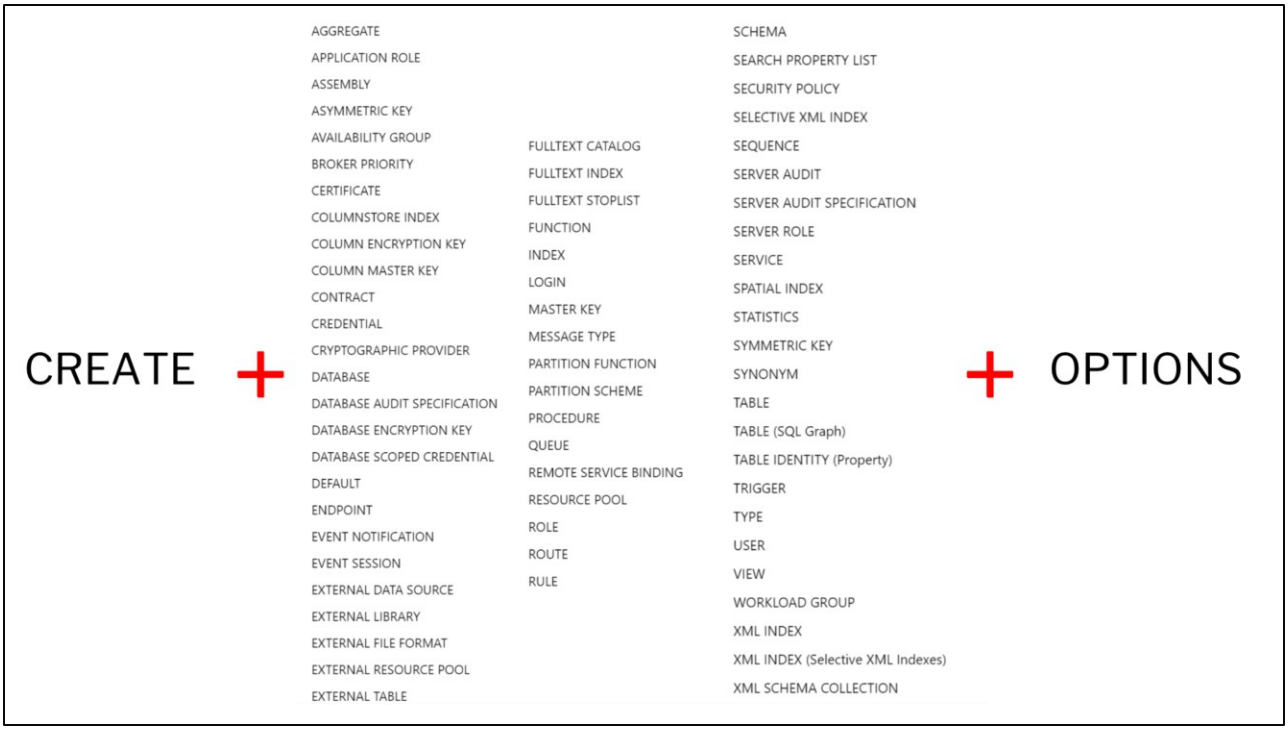


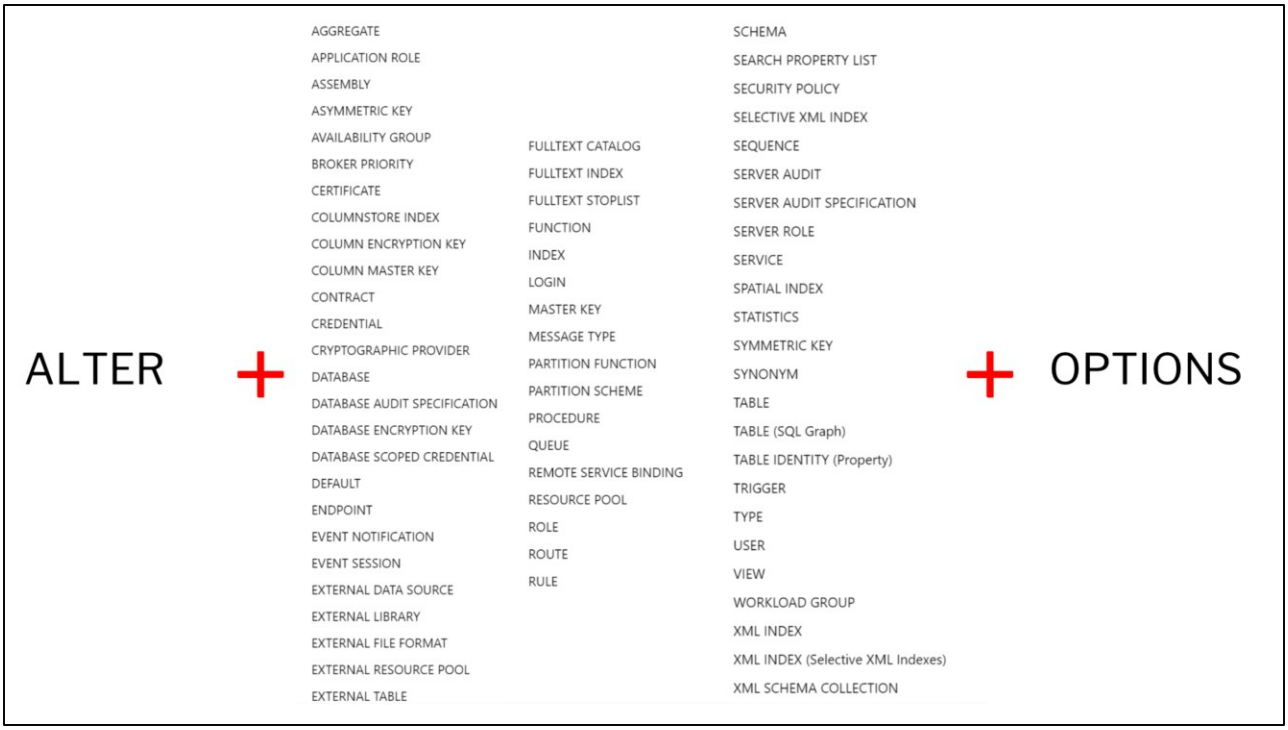
Data Definition Language

CREATE, ALTER, DROP



You can find each command detailed here: https://docs.microsoft.com/en-us/sql/t-sql/statements/statements?view=sql-server-ver15


CREATE
+
AGGREGATE
APPLICATION ROLE
ASSEMBLY
ASYMMETRIC KEY
AVAILABILITY GROUP
BROKER PRIORITY
CERTIFICATE
COLUMNSTORE INDEX
COLUMN ENCRYPTION KEY
COLUMN MASTER KEY
CONTRACT
CREDENTIAL
CRYPTOGRAPHIC PROVIDER
DATABASE
DATABASE AUDIT SPECIFICATION
DATABASE ENCRYPTION KEY
DATABASE SCOPED CREDENTIAL
DEFAULT
ENDPOINT
EVENT NOTIFICATION
EVENT SESSION
EXTERNAL DATA SOURCE
EXTERNAL LIBRARY
EXTERNAL FILE FORMAT
EXTERNAL RESOURCE POOL
EXTERNAL TABLE
FULLTEXT CATALOG
FULLTEXT INDEX
FULLTEXT STOPLIST
FUNCTION
INDEX
LOGIN
MASTER KEY
MESSAGE TYPE
PARTITION FUNCTION
PARTITION SCHEME
PROCEDURE
QUEUE
REMOTE SERVICE BINDING
RESOURCE POOL
ROLE
ROUTE
RULE
SCHEMA
SEARCH PROPERTY LIST
SECURITY POLICY
SELECTIVE XML INDEX
SEQUENCE
SERVER AUDIT
SERVER AUDIT SPECIFICATION
SERVER ROLE
SERVICE
SPATIAL INDEX
STATISTICS
SYMMETRIC KEY
SYNONYM
TABLE
TABLE (SQL Graph)
TABLE IDENTITY (Property)
TRIGGER
TYPE
USER
VIEW
WORKLOAD GROUP
XML INDEX
XML INDEX (Selective XML Indexes)
XML SCHEMA COLLECTION
+
OPTIONS


ALTER
+
AGGREGATE
APPLICATION ROLE
ASSEMBLY
ASYMMETRIC KEY
AVAILABILITY GROUP
BROKER PRIORITY
CERTIFICATE
COLUMNSTORE INDEX
COLUMN ENCRYPTION KEY
COLUMN MASTER KEY
CONTRACT
CREDENTIAL
CRYPTOGRAPHIC PROVIDER
DATABASE
DATABASE AUDIT SPECIFICATION
DATABASE ENCRYPTION KEY
DATABASE SCOPED CREDENTIAL
DEFAULT
ENDPOINT
EVENT NOTIFICATION
EVENT SESSION
EXTERNAL DATA SOURCE
EXTERNAL LIBRARY
EXTERNAL FILE FORMAT
EXTERNAL RESOURCE POOL
EXTERNAL TABLE
FULLTEXT CATALOG
FULLTEXT INDEX
FULLTEXT STOPLIST
FUNCTION
INDEX
LOGIN
MASTER KEY
MESSAGE TYPE
PARTITION FUNCTION
PARTITION SCHEME
PROCEDURE
QUEUE
REMOTE SERVICE BINDING
RESOURCE POOL
ROLE
ROUTE
RULE
SCHEMA
SEARCH PROPERTY LIST
SECURITY POLICY
SELECTIVE XML INDEX
SEQUENCE
SERVER AUDIT
SERVER AUDIT SPECIFICATION
SERVER ROLE
SERVICE
SPATIAL INDEX
STATISTICS
SYMMETRIC KEY
SYNONYM
TABLE
TABLE (SQL Graph)
TABLE IDENTITY (Property)
TRIGGER
TYPE
USER
VIEW
WORKLOAD GROUP
XML INDEX
XML INDEX (Selective XML Indexes)
XML SCHEMA COLLECTION
+
OPTIONS

DROP [IF EXISTS] +

- AGGREGATE
- APPLICATION ROLE
- ASSEMBLY
- ASYMMETRIC KEY
- AVAILABILITY GROUP
- BROKER PRIORITY
- CERTIFICATE
- COLUMNSTORE INDEX
- COLUMN ENCRYPTION KEY
- COLUMN MASTER KEY
- CONTRACT
- CREDENTIAL
- CRYPTOGRAPHIC PROVIDER
- DATABASE
- DATABASE AUDIT SPECIFICATION
- DATABASE ENCRYPTION KEY
- DATABASE SCOPED CREDENTIAL
- DEFAULT
- ENDPOINT
- EVENT NOTIFICATION
- EVENT SESSION
- EXTERNAL DATA SOURCE
- EXTERNAL LIBRARY
- EXTERNAL FILE FORMAT
- EXTERNAL RESOURCE POOL
- EXTERNAL TABLE
- FULLTEXT CATALOG
- FULLTEXT INDEX
- FULLTEXT STOPLIST
- FUNCTION
- INDEX
- LOGIN
- MASTER KEY
- MESSAGE TYPE
- PARTITION FUNCTION
- PARTITION SCHEME
- PROCEDURE
- QUEUE
- REMOTE SERVICE BINDING
- RESOURCE POOL
- ROLE
- ROUTE
- RULE
- SCHEMA
- SEARCH PROPERTY LIST
- SECURITY POLICY
- SELECTIVE XML INDEX
- SEQUENCE
- SERVER AUDIT
- SERVER AUDIT SPECIFICATION
- SERVER ROLE
- SERVICE
- SPATIAL INDEX
- STATISTICS
- SYMMETRIC KEY
- SYNONYM
- TABLE
- TABLE (SQL Graph)
- TABLE IDENTITY (Property)
- TRIGGER
- TYPE
- USER
- VIEW
- WORKLOAD GROUP
- XML INDEX
- XML INDEX (Selective XML Indexes)
- XML SCHEMA COLLECTION

CREATE OR ALTER

- Since SQL Server 2016 we have this option (for Functions, Stored Procedures, Triggers and Views)

- In case the object exists, it will be modified and if not, it will be created



Data Control Language

GRANT, DENY, REVOKE



You can find each command detailed here, in the Permission Section:
https://docs.microsoft.com/en-us/sql/t-sql/statements/statements?view=sql-server-ver15

```
GRANT { ALL [ PRIVILEGES ] }
      | permission [ ( column [ ,...n ] ) ] [ ,...n ]
      [ ON [ class :: ] securable ] TO principal [ ,...n ]
      [ WITH GRANT OPTION ] [ AS principal ]
```

```
DENY   { ALL [ PRIVILEGES ] }
     | <permission>  [ ( column [ ,...n ] ) ] [ ,...n ]
    [ ON [ <class> :: ] securable ]
    TO principal [ ,...n ]
    [ CASCADE] [ AS principal ]
[;]

<permission> ::=
{ see the tables below }

<class> ::=
{ see the tables below }
```

```
REVOKE [ GRANT OPTION FOR ]
      {
        [ ALL [ PRIVILEGES ] ]
        |
                permission [ ( column [ ,...n ] ) ] [ ,...n ]
      }
      [ ON [ class :: ] securable ]
      { TO | FROM } principal [ ,...n ]
      [ CASCADE] [ AS principal ]
```

Data Manipulation Language

SELECT, INSERT, UPDATE, DELETE, MERGE



You can find each command detailed here: https://docs.microsoft.com/en-us/sql/t-sql/queries/queries?view=sql-server-ver15

```
<SELECT statement> ::=
    [ WITH { [ XMLNAMESPACES ,] [ <common_table_expression> [,...n] ] } ]
    <query_expression>
    [ ORDER BY { order_by_expression | column_position [ ASC | DESC ] }
  [ ,...n ] ]
    [ <FOR Clause>]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
<query_expression> ::=
    { <query_specification> | ( <query_expression> ) }
    [  { UNION [ ALL ] | EXCEPT | INTERSECT }
        <query_specification> | ( <query_expression> ) [...n ] ]
<query_specification> ::=
SELECT [ ALL | DISTINCT ]
    [TOP ( expression ) [PERCENT] [ WITH TIES ] ]
    < select_list >
    [ INTO new_table ]
    [ FROM { <table_source> } [ ,...n ] ]
    [ WHERE <search_condition> ]
    [ <GROUP BY> ]
    [ HAVING < search_condition > ]
```

```
[ WITH <common_table_expression> [ ,...n ] ]
INSERT
{
       [ TOP ( expression ) [ PERCENT ] ]
       [ INTO ]
       { <object> | rowset_function_limited
         [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
       }
   {
       [ ( column_list ) ]
       [ <OUTPUT Clause> ]
       { VALUES ( { DEFAULT | NULL | expression } [ ,...n ] ) [ ,...n     ]
       | derived_table
       | execute_statement
       | <dml_table_source>
       | DEFAULT VALUES
       }
   }
}
[;]

<object> ::=
{
    [ server_name . database_name . schema_name .
      | database_name .[ schema_name ] .
      | schema_name .
    ]
  table_or_view_name
}

<dml_table_source> ::=
    SELECT <select_list>
    FROM ( <dml_statement_with_output_clause> )
      [AS] table_alias [ ( column_alias [ ,...n ] ) ]
    [ WHERE <search_condition> ]
       [ OPTION ( <query_hint> [ ,...n ] ) ]
```

```
BULK INSERT
   { database_name.schema_name.table_or_view_name | schema_name.table_or_view_name | table_or_view_name }
      FROM 'data_file'
     [ WITH
      (
    [ [ , ] BATCHSIZE = batch_size ]
    [ [ , ] CHECK_CONSTRAINTS ]
    [ [ , ] CODEPAGE = { 'ACP' | 'OEM' | 'RAW' | 'code_page' } ]
    [ [ , ] DATAFILETYPE =
       { 'char' | 'native'| 'widechar' | 'widenative' } ]
    [ [ , ] DATASOURCE = 'data_source_name' ]
    [ [ , ] ERRORFILE = 'file_name' ]
    [ [ , ] ERRORFILE_DATA_SOURCE = 'data_source_name' ]
    [ [ , ] FIRSTROW = first_row ]
    [ [ , ] FIRE_TRIGGERS ]
    [ [ , ] FORMATFILE_DATASOURCE = 'data_source_name' ]
    [ [ , ] KEEPIDENTITY ]
    [ [ , ] KEEPNULLS ]
    [ [ , ] KILOBYTES_PER_BATCH = kilobytes_per_batch ]
    [ [ , ] LASTROW = last_row ]
    [ [ , ] MAXERRORS = max_errors ]
    [ [ , ] ORDER ( { column [ ASC | DESC ] } [ ,...n ] ) ]
    [ [ , ] ROWS_PER_BATCH = rows_per_batch ]
    [ [ , ] ROWTERMINATOR = 'row_terminator' ]
    [ [ , ] TABLOCK ]

    -- input file format options
    [ [ , ] FORMAT = 'CSV' ]
    [ [ , ] FIELDQUOTE = 'quote_characters']
    [ [ , ] FORMATFILE = 'format_file_path' ]
    [ [ , ] FIELDTERMINATOR = 'field_terminator' ]
    [ [ , ] ROWTERMINATOR = 'row_terminator' ]
     )]
```

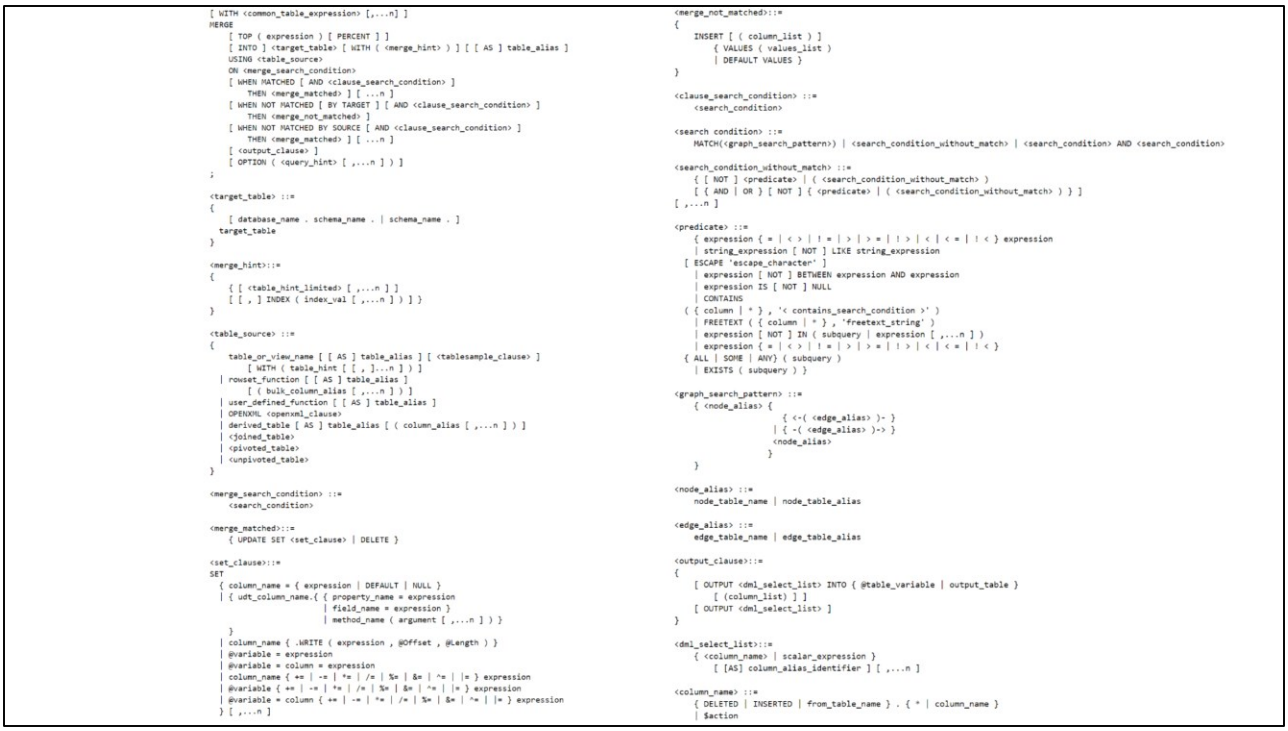
```
[ WITH <common_table_expression> [...n] ]
UPDATE
    [ TOP ( expression ) [ PERCENT ] ]
    { { table_alias | <object> | rowset_function_limited
        [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
      }
      | @table_variable
    }
    SET
        { column_name = { expression | DEFAULT | NULL }
          | { udt_column_name.{ { property_name = expression
                                | field_name = expression }
                                | method_name ( argument [ ,...n ] )
                              }
          }
          | column_name { .WRITE ( expression , @Offset , @Length ) }
          | @variable = expression
          | @variable = column = expression
          | column_name { += | -= | *= | /= | %= | &= | ^= | |= } expression
          | @variable { += | -= | *= | /= | %= | &= | ^= | |= } expression
          | @variable = column { += | -= | *= | /= | %= | &= | ^= | |= } expression
        } [ ,...n ]

    [ <OUTPUT Clause> ]
    [ FROM{ <table_source> } [ ,...n ] ]
    [ WHERE { <search_condition>
            | { [ CURRENT OF
                  { { [ GLOBAL ] cursor_name }
                      | cursor_variable_name
                  }
                ]
              }
            }
    ]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
[ ; ]

<object> ::=
{
    [ server_name . database_name . schema_name .
    | database_name .[ schema_name ] .
    | schema_name .
    ]
    table_or_view_name}
```

```
[ WITH <common_table_expression> [ ,...n ] ]
DELETE
    [ TOP ( expression ) [ PERCENT ] ]
    [ FROM ]
    { { table_alias
      | <object>
      | rowset_function_limited
      [ WITH ( table_hint_limited [ ...n ] ) ] }
      | @table_variable
    }
    [ <OUTPUT Clause> ]
    [ FROM table_source [ ,...n ] ]
    [ WHERE { <search_condition>
            | { [ CURRENT OF
                  { { [ GLOBAL ] cursor_name }
                      | cursor_variable_name
                  }
                ]
              }
            }
    ]
    [ OPTION ( <Query Hint> [ ,...n ] ) ]
[; ]

<object> ::=
{
    [ server_name.database_name.schema_name.
      | database_name. [ schema_name ] .
      | schema_name.
    ]
    table_or_view_name
}
```



```
[ WITH <common_table_expression> [,...n] ]
MERGE
    [ TOP ( expression ) [ PERCENT ] ]
    [ INTO ] <target_table> [ WITH ( <merge_hint> ) ] [ [ AS ] table_alias ]
    USING <table_source>
    ON <merge_search_condition>
    [ WHEN MATCHED [ AND <clause_search_condition> ]
        THEN <merge_matched> ] [ ...n ]
    [ WHEN NOT MATCHED [ BY TARGET ] [ AND <clause_search_condition> ]
        THEN <merge_not_matched> ]
    [ WHEN NOT MATCHED BY SOURCE [ AND <clause_search_condition> ]
        THEN <merge_matched> ] [ ...n ]
    [ <output_clause> ]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
;

<target_table> ::=
{
    [ database_name . schema_name . | schema_name . ]
  target_table
}

<merge_hint>::=
{
    { [ <table_hint_limited> [ ,...n ] ]
    [ [ , ] INDEX ( index_val [ ,...n ] ) ] }
}

<table_source> ::=
{
    table_or_view_name [ [ AS ] table_alias ] [ <tablesample_clause> ]
        [ WITH ( table_hint [ [ , ]...n ] ) ]
  | rowset_function [ [ AS ] table_alias ]
        [ ( bulk_column_alias [ ,...n ] ) ]
  | user_defined_function [ [ AS ] table_alias ]
  | OPENXML <openxml_clause>
  | derived_table [ AS ] table_alias [ ( column_alias [ ,...n ] ) ]
  | <joined_table>
  | <pivoted_table>
  | <unpivoted_table>
}

<merge_search_condition> ::=
    <search_condition>

<merge_matched>::=
    { UPDATE SET <set_clause> | DELETE }

<set_clause>::=
SET
  { column_name = { expression | DEFAULT | NULL }
  | { udt_column_name.{ { property_name = expression
                        | field_name = expression }
                        | method_name ( argument [ ,...n ] ) }
    }
  | column_name { .WRITE ( expression , @Offset , @Length ) }
  | @variable = expression
  | @variable = column = expression
  | column_name { += | -= | *= | /= | %= | &= | ^= | |= } expression
  | @variable { += | -= | *= | /= | %= | &= | ^= | |= } expression
  | @variable = column { += | -= | *= | /= | %= | &= | ^= | |= } expression
  } [ ,...n ]

<merge_not_matched>::=
{
    INSERT [ ( column_list ) ]
        { VALUES ( values_list )
        | DEFAULT VALUES }
}

<clause_search_condition> ::=
    <search_condition>

<search condition> ::=
    MATCH(<graph_search_pattern>) | <search_condition_without_match> | <search_condition> AND <search_condition>

<search_condition_without_match> ::=
    { [ NOT ] <predicate> | ( <search_condition_without_match> )
    [ { AND | OR } [ NOT ] { <predicate> | ( <search_condition_without_match> ) } ]
[ ,...n ]

<predicate> ::=
    { expression { = | < > | ! = | > | > = | ! > | < | < = | ! < } expression
    | string_expression [ NOT ] LIKE string_expression
  [ ESCAPE 'escape_character' ]
    | expression [ NOT ] BETWEEN expression AND expression
    | expression IS [ NOT ] NULL
    | CONTAINS
  ( { column | * } , '< contains_search_condition >' )
    | FREETEXT ( { column | * } , 'freetext_string' )
    | expression [ NOT ] IN ( subquery | expression [ ,...n ] )
    | expression { = | < > | ! = | > | > = | ! > | < | < = | ! < }
  { ALL | SOME | ANY} ( subquery )
    | EXISTS ( subquery ) }

<graph_search_pattern> ::=
    { <node_alias> {
                      { <-( <edge_alias> )- }
                    | { -( <edge_alias> )-> }
                    <node_alias>
                   }
    }

<node_alias> ::=
    node_table_name | node_table_alias

<edge_alias> ::=
    edge_table_name | edge_table_alias

<output_clause>::=
{
    [ OUTPUT <dml_select_list> INTO { @table_variable | output_table }
        [ (column_list) ] ]
    [ OUTPUT <dml_select_list> ]
}

<dml_select_list>::=
    { <column_name> | scalar_expression }
        [ [AS] column_alias_identifier ] [ ,...n ]

<column_name> ::=
    { DELETED | INSERTED | from_table_name } . { * | column_name }
    | $action
```

Merge takes 2 tables, let them be source and target, and tries to combine those into the target table based on the matching conditions.

Order when querying

[WITH]
SELECT
[INTO]
[FROM]
[[CROSS]|[INNER]|[LEFT|RIGHT|FULL OUTER]] JOIN
[WHERE]
[GROUP BY]
[HAVING]
[ORDER BY]



Special Instructions

User-Defined Function

- Like functions in programming languages, SQL Server user-defined functions are routines that accept parameters, perform an action, such as a complex calculation, and return the result of that action as a value.

- The return value can either be a single scalar value or a result set.



Functions

- READ-ONLY
- Faster execution
- Can reduce network traffic



Stored Procedures

- A stored procedure in SQL Server is a group of one or more Transact-SQL statements or a reference to a Microsoft .NET Framework common runtime language (CLR) method. Procedures resemble constructs because they can:
  - Accept input parameters and return multiple values.
  - Contain programming statements that perform operations in the database. These include calling other procedures.
  - Return a status value to a calling program to indicate success or failure (and the reason for failure).



Stored Procedures

- Reduced server/client network traffic
- Stronger security
- Reuse of code
- Improved performance



Triggers

- Event-driven types of instructions that are used to the integrity of the DB
- DDL: FOR
- DML: AFTER/INSTEAD OF



DDL Triggers

- DDL triggers fire in response to a variety of Data Definition Language (DDL) events. (CREATE, ALTER, DROP, GRANT, DENY, REVOKE or UPDATE STATISTICS)

- Certain system stored procedures that perform DDL-like operations can also fire DDL triggers.



DML Triggers

- DML triggers is a special type of stored procedure that automatically takes effect when a data manipulation language (DML) event takes place that affects the table or view defined in the trigger. (INSERT, UPDATE, or DELETE).

- DML triggers can be used to enforce business rules and data integrity, query other tables, and include complex Transact-SQL statements.



There will be always “entire set” operations, meaning that the trigger will fire only once per table change. Each time there will be 2 tables:

- Inserted, contains nothing if we are in the DELETE case
- Deleted, contains nothing if we are in the INSERT case

SQL Server 2017

Installing



SQL Server 2017

Working with T-SQL

